and you're checking all of them against `max_hazard_pointers` stored hazard pointers. Ouch! There has to be a better way.

Better Reclamation Strategies Using Hazard Pointers

Of course, there *is* a better way. What I've shown here is a simple and naïve implementation of hazard pointers to help explain the technique. The first thing you can do is trade memory for performance. Rather than checking every node on the reclamation list every time you call `pop()`, you don't try to reclaim any nodes at all unless there are more than `max_hazard_pointers` nodes on the list. That way you're guaranteed to be able to reclaim at least one node. If you just wait until there are `max_hazard_pointers+1` nodes on the list, you're not much better off. Once you get to `max_hazard_pointers` nodes, you'll be trying to reclaim nodes for most calls to `pop()`, so you're not doing much better. But if you wait until there are `2*max_hazard_pointers` nodes on the list, you're guaranteed to be able to reclaim at least `max_hazard_pointers` nodes, and it will then be at least `max_hazard_pointers` calls to `pop()` before you try to reclaim any nodes again. This is much better. Rather than checking around `max_hazard_pointers` nodes every call to `push()` (and not necessarily reclaiming any), you're checking `2*max_hazard_pointers` nodes every `max_hazard_pointers` calls to `pop()` and reclaiming at least `max_hazard_pointers` nodes. That's effectively two nodes checked for every `pop()`, one of which is reclaimed.

Even this has a downside (beyond the increased memory usage): you now have to count the nodes on the reclamation list, which means using an atomic count, and you still have multiple threads competing to access the reclamation list itself. If you have memory to spare, you can trade increased memory usage for an even better reclamation scheme: each thread keeps its own reclamation list in a thread-local variable. There's thus no need for atomic variables for the count or the list access. Instead, you have `max_hazard_pointers*max_hazard_pointers` nodes allocated. If a thread exits before all its nodes have been reclaimed, they can be stored